

SAG: SQL-RETRIEVAL AUGMENTED GENERATION WITH QUERY-TIME DYNAMIC HYPEREDGES

**Yuchao Wu*, Junqin Li, XingCheng Liang, Yongjie Chen, Yinghao Liang
Linyuan Mo, Guanxian Li**

Zleap AI

{jomy, junqing, lensen, jinzhoulawen, leo}@zleap.com

{mo-linyuan, li-guanxian}@foxmail.com

ABSTRACT

Retrieval-Augmented Generation (RAG) offers an effective approach for large language models to access external knowledge. However, existing methods rely on dense similarity retrieval and face inherent limitations in handling structured constraints and multi-hop reasoning. Incorporating knowledge graphs partially alleviates these issues, but at the cost of semantic fragmentation, high maintenance overhead, and difficult incremental updates. This paper introduces SAG (SQL-Retrieval Augmented Generation), a structured architecture for retrieval and agent systems. Instead of pre-building a global static graph, SAG converts each chunk into one semantically complete event and a set of indexing entities, then uses SQL join queries to dynamically link events that share entities into local hyperedges, constructing, at query time, a dynamically instantiated local index structure. This design avoids the need for global graph rebuilding and ongoing maintenance; the system naturally supports incremental writes, concurrent processing, and continuous scaling through its reliance on standard database infrastructure. Across HotpotQA, 2WikiMultiHop, and MuSiQue, three standard multi-hop benchmarks, SAG achieves the best results on 8 out of 9 Recall@K metrics, reaching 80.0% Recall@5 on MuSiQue, the benchmark with the highest multi-hop reasoning demands. SAG has also been deployed at a production scale of hundreds of millions of data items, with online retrieval latency kept within seconds. Project site and code are available at <https://github.com/Zleap-AI/SAG-Benchmark>.

1 INTRODUCTION

As large language model capabilities continue to advance, the bottleneck in agent systems is shifting from model capability to data infrastructure. Faced with ever-growing corpora, cross-system associations, and evolving states, agents require not one-shot static retrieval but rather a retrieval infrastructure capable of continuously ingesting incremental data and supporting multi-step associative queries. The dominant approach in retrieval-augmented generation (RAG) today splits documents into chunks, maps them to a vector space, and retrieves the most similar chunks at query time (Lewis et al., 2020; Karpukhin et al., 2020), and performs robustly on tasks such as open-domain QA. However, agents often require multi-step sequential retrieval, where errors at each step accumulate and amplify along the reasoning chain. The retrieval infrastructure therefore needs not merely a higher single-shot recall rate, but the ability to reliably organize evidence across multi-hop queries.

Existing methods address this challenge along two main directions, each with its own limitations. The first, dense retrieval, is at its core semantic-similarity matching; it excels at retrieving semantically proximate passages but struggles to recover explicit associative chains between entities, let alone organize evidence scattered across multiple documents into structured evidence chains (Yang et al., 2018; Trivedi et al., 2022; Mavi et al., 2024). This limitation becomes particularly pronounced when queries involve temporal constraints, entity roles, or multi-step dependencies. The second direction, structure-augmented methods, instead builds knowledge graphs or hierarchical summaries offline from documents to explicitly represent entity relationships (Edge et al., 2024; Gutiérrez et al., 2025).

*Correspondence to: jomy@zleap.com.

But explicit structure comes at a cost. Triple extraction, entity merging, and relation normalization each introduce errors at successive stages, and construction costs are high. As data evolves, the cost of maintaining a global graph may even exceed that of building it. More critically, these carefully constructed offline structures often degrade into flat similarity matching at the node or summary level at query time, a systematic decoupling between offline structure and online recall (see Section 2 for details).

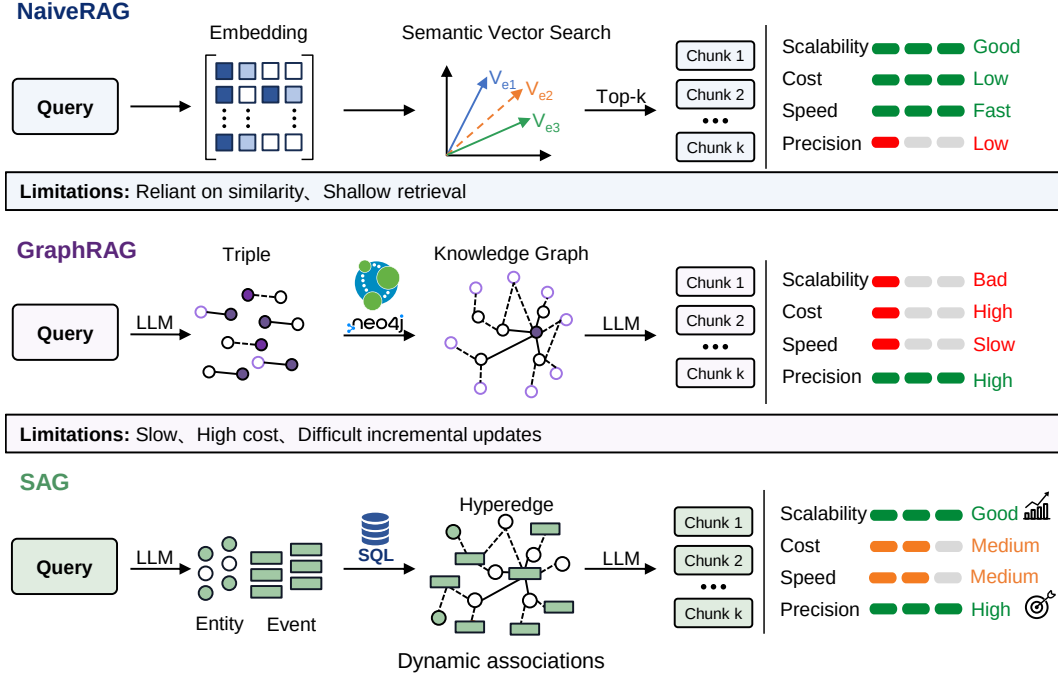


Figure 1: Process and performance comparison of three RAG paradigms. NaiveRAG retrieves top-k chunks via dense vector similarity, offering good scalability, high speed, and low cost, but precision is limited by shallow semantic matching. GraphRAG extracts triples and builds a knowledge graph offline using an LLM; it enhances evidence organization capability, but incurs high construction cost and is difficult to update incrementally. SAG extracts events and entities with an LLM and activates dynamic hyperedge structures at query time via SQL; it balances retrieval quality, structural capability, and system overhead, while naturally supporting append-only incremental updates.

Our central claim is that, for queries involving structural constraints and multi-hop associations, retrieval should be governed neither entirely by dense similarity nor by offline pre-constructed static graphs. SAG transforms documents into event-entity indices, where each chunk corresponds to an event that preserves complete semantics and a set of entities serving an indexing function, together defining a latent hyperedge (see Figure 1 for a comparison of the three paradigms). At query time, SQL drives deterministic event-entity association and local hyperedge activation, this structural path is combined with vector retrieval into a unified pipeline, with the LLM performs only the final selection over a compressed candidate set. Because hyperedges are not pre-constructed but dynamically instantiated around the current query, the system does not depend on a static graph structure and requires no global recomputation.

In summary, the main contributions of this paper are: (1) we propose SAG, a structured retrieval architecture that replaces offline static graphs with item-entity indexes and takes SQL-driven retrieval as its core. It unifies three capabilities: structured filtering, semantic expansion, and LLM fine ranking in a single pipeline; (2) we design a query-time dynamic hyperedge organization mechanism, enabling higher-order relations to be dynamically activated within a local candidate space around the current query without prior enumeration, and to be deterministically expanded across multiple hops via SQL joins; (3) we systematically evaluate SAG on three multi-hop benchmarks and, through ablation studies, isolate the respective contributions of event-level semantic preservation, dynamic expansion, LLM usage patterns, and candidate budget; (4) we have deployed SAG in a production

environment at a scale of hundreds of millions of records, demonstrating the engineering feasibility of this framework under continuous incremental writes and online cost constraints.

2 RELATED WORK

2.1 RETRIEVAL-AUGMENTED GENERATION

Retrieval-Augmented Generation (RAG) connects LLMs to external knowledge by recasting knowledge access as a retrieval problem (Lewis et al., 2020; Karpukhin et al., 2020), but the default chunk-vector-top-k pipeline introduces the classic chunking dilemma and fixes retrieval as a step that precedes reasoning; candidates are determined before the model understands the query. Subsequent work introduces adaptive retrieval along two dimensions. Triggering strategies allow the model to autonomously decide when to retrieve (Asai et al., 2024; Jiang et al., 2023b; Jeong et al., 2024), while iterative reasoning interleaves multi-step retrieval with reasoning (Trivedi et al., 2023; Zhuang et al., 2024). These advances focus on *when to retrieve* and *what query to retrieve with*, but once candidates are identified, the organization stage still relies on semantic similarity alone; it cannot perform structured filtering or explicit entity association across candidates.

2.2 STRUCTURE-AUGMENTED RETRIEVAL AND GRAPH-BASED RAG

To compensate for the shortcomings of vector retrieval in multi-hop evidence organization, structure-augmented RAG has developed along three main lines. The first builds a knowledge graph offline from documents, then activates multi-hop subgraphs via graph traversal or ranking; representative works include GraphRAG (Edge et al., 2024), HippoRAG and its successors (Gutiérrez et al., 2024; 2025), along with methods exploring prompt fusion, graph neural network integration, and interactive traversal over existing KGs (Wang et al., 2024; He et al., 2024; Sun et al., 2024; Liu et al., 2025; Mavromatis et al., 2025). The second line organizes documents hierarchically (Sarathi et al., 2024; Huang et al., 2025; Guo et al., 2025; Zhang et al., 2025), among which SiReRAG validates the value of structured indexing for recall on the same three benchmarks used here. The third line performs runtime dynamic structuring, where StructRAG (Li et al., 2025) converts documents into a hybrid knowledge representation at inference time. All three lines face the same trade-off. The structures built offline or at inference time are costly to construct, yet query-time retrieval often degenerates into flat similarity matching over nodes or summaries, a systematic decoupling between carefully designed offline structure and online recall. SAG eliminates this decoupling at the mechanism level by activating local hyperedges at query time via SQL, embedding structural organization into the retrieval execution itself.

2.3 SQL, TABLE-GROUNDED QA, AND STRUCTURED RETRIEVAL INTERFACES

Research on QA over structured data confirms that structured interfaces improve retrieval precision, yet these approaches all presuppose that the underlying structure already exists. Table QA validates the reasoning capability of language models over tables and proposes cross-table-text multi-hop tasks (Herzig et al., 2020; Liu et al., 2022; Chen et al., 2020); text-to-SQL extends this direction to the query side, constructing large-scale cross-domain benchmarks and steadily improving complex SQL generation (Yu et al., 2018; Li et al., 2023; Pourreza & Rafiei, 2023); StructGPT (Jiang et al., 2023a) and ChatDB (Hu et al., 2023) incorporate structured interfaces into LLM systems for iterative interface reasoning and relational-database-style long-term memory, respectively. These approaches either treat SQL as a generation target or simply presuppose that structured data is already in place; they address the question of *how to query an existing structure*. SAG addresses a prior question; it first constructs queryable event-entity tables from unstructured documents through offline event extraction, then drives the main retrieval path of open-domain RAG with SQL’s precise filtering, integrated with vector retrieval into a single pipeline.

2.4 HYPEREDGE AND HIGHER-ORDER RELATIONAL REPRESENTATIONS

Traditional knowledge graphs represent binary relations as triples (h, r, t) , but real-world events often involve multiple parties or dimensions; forcing them into binary relations destroys their overall semantics. Work on hypergraph learning and n-ary relational modeling shows that higher-order

representations preserve original semantic structure better than triple decomposition (Zhou et al., 2006; Feng et al., 2019; Fatemi et al., 2020; Galkin et al., 2020). However, this idea remains underutilized in RAG; mainstream graph-augmented methods still model entities as nodes and binary edges as relations (Edge et al., 2024; Gutiérrez et al., 2025). Recently, HGRAG (Wang et al., 2026) and Graph-R1 (Luo et al., 2026) introduce hypergraphs, but both pre-build hypergraph structures offline, with query-time activation relying on embedding-based approximate matching; as a result, they still incur the cost of maintaining a static structure and suffer from accumulated matching errors. SAG treats each event and its associated entities as a latent hyperedge, dynamically instantiating local hyperedges at query time by using SQL to identify sets of events that share entities, avoiding the offline cost of hypergraph pre-construction while preserving the expressive power of higher-order representations in n-ary event scenarios.

3 SAG

3.1 FRAMEWORK OVERVIEW

Figure 2 presents the complete architecture of SAG, organized into offline and online phases. In the offline phase, each document chunk is transformed into one event and a set of entities, and written synchronously into a SQL database, a vector index, and a full-text index. The online phase then runs three steps in sequence, namely seed retrieval, query-time expansion, and final selection; the detailed design of each step is presented in the following subsections.

The three stages rest on a clear division of labor across modules. SQL handles deterministic filtering and joining, vector retrieval handles semantic expansion for aliases, near-synonymous expressions, and paraphrases, and the LLM is reserved for a small number of high-value semantic decision points. From a process perspective, SAG retrieval consists of three sequential steps. First, locate entry points to relevant events through entities (seed retrieval); then, hop between different events along shared entities to expand the candidate pool (query-time expansion); finally, perform fine-grained selection over the compressed candidate space (LLM reranking).

3.2 EVENT-ENTITY INDEX

SAG builds an event-entity index, not a global knowledge graph. Given a chunk, the indexing stage produces one event e and a set of entities $\mathcal{U}(e)$, together defining a latent hyperedge.

Event is a concise statement of the chunk’s core content, with a one-chunk-to-one-event mapping. The event preserves complete semantics and is not further decomposed into multiple independent triples, thereby avoiding the semantic fragmentation problem inherent in triple extraction while providing the necessary data foundation for query-time dynamic hyperedge activation.

Entities span time, location, person, organization, group, topic, work, product, action, metric, and label, for a total of 11 types. Entities do not carry complete semantics; they serve only as index and expansion points for connecting different events.

Events and entities are not produced by a cascade of sequential extraction steps; they are two parallel structured outputs for the same chunk. Extraction results are written into SQL, establishing many-to-many connections between events and entities; event text and entity text are simultaneously written into vector and full-text indices. One event linked to multiple entities defines a latent hyperedge.

This index deliberately avoids introducing a full entity disambiguation system. It is the event that truly carries semantics; entities serve merely as index points and expansion points. SAG adopts a pragmatic strategy for entity handling, relying on simple string normalization and SQL deduplication; no complete entity merging mechanism is needed for stable operation. By design, SAG’s index layer is not a heavyweight knowledge graph, but a lightweight, appendable semantic index over unstructured documents.

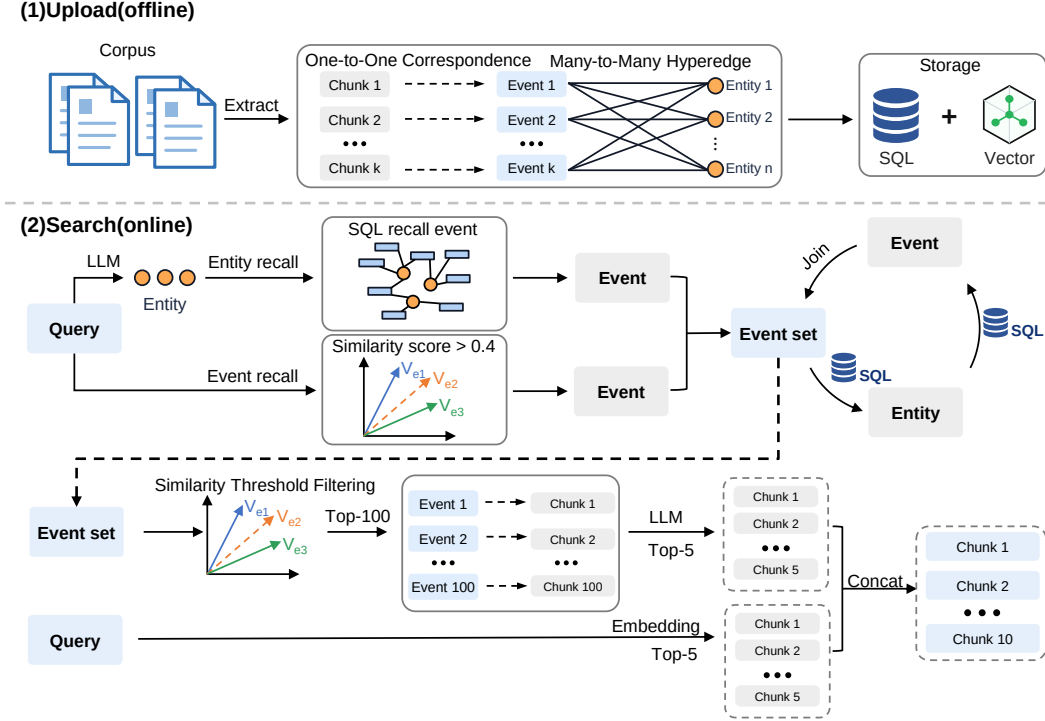


Figure 2: Architecture overview of SAG. In the offline phase, each chunk is transformed into one event and a set of entities, and written into SQL, vector, and full-text indices. In the online phase, the system performs initial recall, then query-time expansion, and finally completes selection over the compressed candidate set.

3.3 SEED RETRIEVAL

Given a query q , SAG constructs the initial candidate event set $\mathcal{E}_R$ through two parallel paths.

Path A: entity-guided structured recall. An LLM identifies key entities in the query text, producing a seed entity set $\mathcal{U}_q = \{u_i\}$. The system performs similarity retrieval over the entity vector index using each seed entity as a query (default threshold 0.9), recalling semantically similar expansion entities to produce an augmented entity set $\hat{\mathcal{U}}_q \supseteq \mathcal{U}_q$. SQL join queries then retrieve all events associated with these entities:

$$\mathcal{E}_R^{\text{entity}} = \{e \mid \exists u \in \hat{\mathcal{U}}_q : \text{SQL-Join}(e, u)\}. \quad (1)$$

Path B: direct event recall via query vector. The system simultaneously performs similarity retrieval over the event index using the query vector, retaining events whose similarity exceeds threshold τ (default 0.4), yielding direct candidates $\mathcal{E}_R^{\text{direct}}$. The two paths are merged to form the initial candidate event set:

$$\mathcal{E}_R = \mathcal{E}_R^{\text{entity}} \cup \mathcal{E}_R^{\text{direct}}. \quad (2)$$

Path A covers structural multi-hop clues through entity association; Path B covers directly relevant events through semantic matching. Together, they constitute the initial candidate recall stage.

3.4 QUERY-TIME EXPANSION AND SELECTION

Query-time expansion broadens the seed retrieval results into a more comprehensive candidate pool. Hyperedges are not explicitly enumerated offline but dynamically instantiated around the candidates of the current query.

Expansion. Starting from $\mathcal{E}_R$, the system performs reverse SQL join queries to extract associated entities, forming an entity frontier (the set of entities newly linked to seed events but not yet explored),

then uses these frontier entities as bridge points to discover new events, expanding the candidate set hop by hop. This process relies solely on SQL joins; multi-hop expansion amounts to relational joins in a database, not PageRank or graph reasoning. Expansion runs for at most H hops (default $H = 1$), with each round only introducing previously unseen entities and events. Denoting the set of events added during expansion as $\mathcal{E}_E$, the full candidate pool is:

$$\mathcal{E}_{\text{cand}} = \mathcal{E}_R \cup \mathcal{E}_E. \quad (3)$$

Coarse ranking. SAG filters the candidate events in $\mathcal{E}_{\text{cand}}$ by similarity to the query vector and retains the top K_{cand} (default 100), denoted $\hat{\mathcal{E}}$.

Dual-path output. The system executes two output paths in parallel. Path A (structural path): the LLM performs final reranking over $\hat{\mathcal{E}}$, selecting the top K_{event} events to obtain $\mathcal{E}^* = \text{Rerank}(\hat{\mathcal{E}}, q, f_{\text{LLM}})$, then maps them back to original chunks to obtain $\mathcal{C}^{\text{event}} = \text{Map}(\mathcal{E}^*)$. Path B (semantic path): direct chunk retrieval using the query vector over the chunk index selects the top K_{direct} chunks to obtain $\mathcal{C}^{\text{direct}} = \text{Embed}_{\text{top-}K}(q)$. The two sets are merged, deduplicated, and the top K_{out} chunks are returned as final evidence:

$$\mathcal{C}_{\text{out}} = \text{TopK}_{K_{\text{out}}}(\mathcal{C}^{\text{event}} \cup \mathcal{C}^{\text{direct}}). \quad (4)$$

3.5 INTERPRETABILITY

This pipeline naturally yields a fully auditable trace.

$$q \rightarrow \mathcal{U}_q \rightarrow \hat{\mathcal{U}}_q \rightarrow \mathcal{E}_R \rightarrow \mathcal{E}_{\text{cand}} \rightarrow \hat{\mathcal{E}} \rightarrow \mathcal{C}_{\text{out}}. \quad (5)$$

Every step in this chain is inspectable. For the query ‘‘Which project did the CTO of the company that acquired Company B later join?’’ the chain unfolds step by step: entities identified from the query $\{\text{Company B, CTO}\}$, alias expansion via entity vector search, SQL join queries connecting to events such as ‘‘Company A acquired Company B’’ and ‘‘Someone joined Project C’’ followed by hop-based aggregation along shared entities, ultimately ranking the corresponding original chunks into the output. An empty result at any link directly pinpoints the failure location: whether an entity was not identified, an expansion produced no new candidates, or a SQL join returned no results. This step-by-step inspectability allows retrieval failures to be localized to specific stages, rather than appearing as an indecomposable end-to-end score.

4 EXPERIMENTS

4.1 DATASETS

We select HotpotQA, 2WikiMultiHopQA, and MuSiQue, three multi-hop benchmarks with progressively greater demands on reasoning chain depth and non-skippability, suitable for evaluating cross-document entity-linking capability at different difficulty levels.

HotpotQA (Yang et al., 2018) primarily contains bridge and comparison questions requiring reasoning across two Wikipedia documents. We adopt the fullwiki setting, requiring the system to retrieve supporting passages from the complete Wikipedia corpus rather than selecting from a pre-filtered candidate set. The dataset provides passage-level supporting fact annotations for each question, and we evaluate recall directly against the gold paragraphs. This dataset primarily assesses cross-document entity-linking capability in two-hop reasoning scenarios.

2WikiMultiHopQA (Ho et al., 2020) contains multi-hop questions spanning multiple Wikipedia documents, covering bridge, comparison, and inference types, and provides explicit reasoning path annotations covering the intermediate paragraphs relied upon at each reasoning step. This dataset requires the system to simultaneously locate both bridging paragraphs and final answer paragraphs, placing greater demands on reasoning across multiple paths.

MuSiQue (Trivedi et al., 2022) constructs multi-hop questions by progressively composing single-hop questions, including samples with up to four reasoning steps. Through rigorous counterfactual

filtering, the dataset ensures every reasoning step is non-skippable, preventing systems from bypassing the full reasoning chain via single-step semantic matching. This property makes MuSiQue the core testbed for SAG’s hyperedge expansion mechanism; any method relying solely on vector similarity faces its greatest challenge here. We use its answerable subset for evaluation.

All three datasets provide passage-level supporting fact annotations, so Recall directly measures whether the system ranks supporting evidence near the top, independent of what the generation model later selects. All experiments are conducted on the development set of each dataset. Table 1 summarizes the basic statistics of the three datasets.

Table 1: Statistics of the evaluation subsets used in this study. For each dataset, we randomly sample 1,000 questions from the official development split. “Passages” refers to the number of passages in the local corpus constructed from the sampled questions’ supporting documents.

	MuSiQue	2Wiki	HotpotQA
Num of queries	1,000	1,000	1,000
Num of passages	11,656	6,119	9,811

4.2 COMPARED METHODS

We compare SAG against HippoRAG 2 (Gutiérrez et al., 2025). HippoRAG 2 first extracts entities and relations offline from documents using an LLM and builds a knowledge graph, then performs multi-hop graph retrieval at query time via Personalized PageRank, representing a leading implementation of the “offline graph construction + global graph ranking” paradigm and forming a direct mechanistic contrast with SAG’s “query-time dynamic hyperedge” approach. We re-run HippoRAG 2 under the same embedding model and LLM configuration as SAG to isolate the effect of underlying model choices. Additionally, to situate SAG within the retrieval quality spectrum, Table 2 includes simple retrievers and large 7B-parameter embedding retrievers as background reference.

4.3 METRICS

We adopt Recall@K as the primary retrieval metric, measuring the proportion of queries for which the system includes at least one piece of supporting evidence among the top K returned passages (any-hit criterion). This metric measures single-step evidence accessibility rather than full multi-hop evidence chain coverage, and therefore tends to overstate performance under strict multi-hop evaluation; we treat it as an approximate measure aligned with practical retrieval utility, rather than a strict guarantee of reasoning chain completeness.

Nevertheless, in agent sequential retrieval scenarios, top-ranked Recall@K is of direct engineering value. When the returned candidates are fewer but more precisely ranked, the context window required for downstream LLM processing is smaller, the invocation cost is lower, and more retrieval budget remains for subsequent steps. We primarily report Recall@2 and Recall@5, with Recall@1 included in ablation to characterize the stability of top-ranked hits.

For HotpotQA and 2WikiMultiHopQA, supporting passages are the gold paragraphs provided by the datasets; for MuSiQue, supporting passages are the set of paragraphs covered by the annotated reasoning path. Retrieval is considered successful if at least one piece of supporting evidence appears among the top K returned passages.

4.4 IMPLEMENTATION DETAILS

LLM configuration. We use Qwen3.6-Flash (Qwen Team, 2025) for event summarization and entity extraction in the offline phase, and candidate reranking in the online phase. Offline extraction runs in batch mode; online reranking operates only over the compressed candidate set, so the number of LLM calls stays small. The query entity extraction model f_{entity} is likewise driven by Qwen3.6-Flash with few-shot prompting, responsible for identifying entity clues from the query; this is the sole lightweight LLM call in the online phase. The event summarization model f_{event} performs chunk-to-event transformation and entity extraction in batch mode.

Embedding configuration. The default embedding model is BGE-Large-EN-v1.5 (Xiao et al., 2023). We additionally conduct a comparison experiment using NV-Embed-v2 (Lee et al., 2024) on MuSiQue. SAG uses MySQL as the structured storage backend and Elasticsearch as the full-text search engine; vector similarity retrieval for events and entities is also implemented via Elasticsearch’s dense vector capability rather than introducing a separate dedicated vector database.

Hyperparameter settings. Unless otherwise noted, SAG adopts the following default configuration. Expansion hops $H = 1$, initial seed recall budget $K_{\text{seed}} = 50$, entity frontier pruning budget 50, candidate events passed to LLM $K_{\text{cand}} = 100$. The system ultimately returns $K_{\text{out}} = 10$ chunks, of which $K_{\text{event}} = 5$ come from LLM reranking of candidate events and $K_{\text{direct}} = 5$ from direct query-vector retrieval, the two streams are merged and deduplicated before output. Entity types cover time, location, person, organization, group, topic, work, product, action, metric, and label, 11 types in total, and can be extended by domain.

5 RESULTS AND ANALYSIS

5.1 MAIN RESULTS

Table 2 reports Recall@2/5 on the three multi-hop benchmarks. Under the unified underlying configuration, SAG achieves an average Recall@2/5 of 79.3%/88.2%, exceeding HippoRAG 2’s 68.2%/83.3% by 11.1/4.9 percentage points. Except for Recall@5 on 2WikiMultiHop, SAG achieves the best result on all remaining metrics.

SAG’s advantage is most pronounced on MuSiQue, which features the longest reasoning chains. MuSiQue contains up to 4-hop reasoning, and every reasoning step is non-skippable, preventing systems from bypassing the full reasoning chain via single-step semantic matching. SAG achieves Recall@5 of 80.0% vs. HippoRAG 2’s 65.1%, with an even larger gap at Recall@2 (64.1% vs. 49.5%). This pattern is consistent with the mechanistic difference between the two approaches. SAG uses SQL joins to deterministically expand along shared entities, with the expansion path at each hop being explicit and traceable. HippoRAG 2 propagates scores across a global graph via Personalized PageRank; the scores of distant nodes decay under the damping factor, with the influence of noisy nodes compounding through indirect paths as the number of hops grows, an effect that becomes more consequential in scenarios with longer reasoning chains.

Table 2: Recall@2/5 of SAG and various baselines on three datasets under a unified underlying configuration (BGE-Large-EN-v1.5 + Qwen3.6-Flash). Values marked with * are taken from literature reports and serve only as background reference.

	Multi-hop Datasets			
	MuSiQue	2Wiki	HotpotQA	Avg
<i>Simple Baselines</i>				
Contriever*	34.8 / 46.6	46.6 / 57.5	58.4 / 75.3	46.6 / 59.8
BM25*	32.4 / 43.5	55.3 / 65.3	57.3 / 74.8	48.3 / 61.2
GTR (T5-based)*	37.4 / 49.1	60.2 / 67.9	59.3 / 73.9	52.3 / 63.6
<i>Large Embedding Models</i>				
BGE-Large-EN-v1.5	41.6 / 56.2	61.6 / 69.0	76.0 / 88.8	59.7 / 71.3
GTE-Qwen2-7B-Instruct*	48.1 / 63.6	66.7 / 74.8	75.8 / 89.1	63.5 / 75.8
GritLM-7B*	49.7 / 65.9	67.3 / 76.0	79.2 / 92.4	65.4 / 78.1
NV-Embed-v2 (7B)*	52.7 / 69.7	67.1 / 76.5	84.1 / 94.5	68.0 / 80.2
<i>Graph-based Methods</i>				
HippoRAG 2 (BGE-Large-EN-v1.5)	49.5 / 65.1	76.6 / 90.4	78.4 / 94.4	68.2 / 83.3
SAG (Ours)	64.1 / 80.0	82.3 / 88.0	91.6 / 96.5	79.3 / 88.2

On HotpotQA, which involves shorter, predominantly 2-hop reasoning chains, the Recall@5 gap between the two methods narrows (96.5% vs. 94.4%), while the Recall@2 gap still reaches 13.2 percentage points (91.6% vs. 78.4%). The narrower Recall@5 gap is consistent with the mechanism

described above; when the reasoning chain has only 2 hops, the attenuation effect of global graph propagation has yet to fully manifest, and both methods can achieve coverage at deeper ranks. SAG retains its advantage at top ranks, indicating that the hyperedge structure delivers stable improvements in ranking quality. In multi-step agent retrieval, single-step errors accumulate and amplify along the reasoning chain, so the quality of the top-ranked hits directly determines the retrieval space and error margin available to later steps.

On 2WikiMultiHop, SAG’s Recall@5 is 88.0%, slightly below HippoRAG 2’s 90.4%, though Recall@2 still leads at 82.3% (vs. 76.6% for HippoRAG 2), a finding that demands a direct analysis. Some questions in 2WikiMultiHop involve extremely long entity chains. When a bridging entity appears at very low frequency in the corpus, SAG’s fixed pruning budget (entity frontier pruning budget of 50) may truncate it early in expansion, preventing distal events that depend on it from entering the candidate pool. HippoRAG 2’s global PageRank, by contrast, can reach these low-frequency nodes through full-graph propagation. This indicates that SAG’s current pruning strategy has a systematic blind spot for low-frequency bridging entities, which is a specific weakness requiring future improvement. The fact that SAG’s Recall@2 still leads in this setting suggests that this weakness predominantly affects tail recall rather than head hits.

5.2 ABLATION STUDY

We conduct ablation on SAG’s key design choices using MuSiQue, as it provides the strictest test of multi-hop reasoning due to its mandatory intermediate steps and counterfactual filtering, making design differences most discriminable. All ablations retain SAG’s default settings, varying only the single variable under examination.

Hyperedge vs. triples. In the triple-based variant of SAG, each event is decomposed into a set of (subject, predicate, object) triples, with each triple independently indexed and participating in SQL joins; the rest of the pipeline remains unchanged. As shown in Table 3, the triple-based SAG achieves Recall@5 of 77.1%, vs. 80.0% for the hyperedge version, both above HippoRAG 2’s 65.1%. HippoRAG 2’s core relational unit is the entity–relation–entity triple, placing it in the same representational class as triple-based SAG for the purpose of this comparison. Under equal representational conditions, the SAG pipeline still leads by 12 percentage points, showing that the gap comes from the architecture rather than the representation. SAG uses SQL joins to deterministically expand along entity paths hop by hop, with clear and controllable coverage at each step; HippoRAG 2 layers propagation across the full graph via Personalized PageRank, so score decay at distant nodes and interference from noisy nodes both grow with the number of hops. On top of this, the hyperedge structure contributes a further 2.9 percentage points. A hyperedge compresses multiple entities within a single record, allowing a single SQL join to activate all associated paths at once; the triple representation, with each record covering only two endpoints, requires more hops for equivalent coverage, and each hop introduces additional pruning loss. Taken together, SAG’s gains have two components: the pipeline architecture opens a systematic gap over comparable methods, and the hyperedge representation tightens the recall boundary further.

Table 3: Ablation comparison of event-level indexing vs. triple-decomposition indexing (MuSiQue).

Configuration	MuSiQue			
	R@1	R@2	R@5	R@10
Triple	35.6	61.5	77.1	81.2
Hyperedge	36.2	64.1	80.0	83.4

Contribution of query-time expansion. Setting expansion hops H to 0 disables expansion while keeping everything else unchanged. As Table 4 shows, disabling expansion causes Recall@5 to drop from 80.0% to 69.4%, while Recall@1 remains nearly unchanged (36.2% vs. 35.7%). This pattern reveals what expansion actually contributes: it adds evidence that vector recall cannot surface on its own, rather than improving the ranking of candidates already in the pool. The events introduced by expansion, accessed through shared-entity paths, lack direct semantic overlap with the query, so vector similarity cannot reliably place them at top ranks; hence Recall@1 is largely unaffected. Yet these candidates are precisely the critical intermediate evidence in multi-hop reasoning chains, and their absence causes a substantial drop in Recall@5.

Table 4: Ablation study on the number of expansion hops in dynamic hyperedge construction (MuSiQue).

Hops	MuSiQue			
	R@1	R@2	R@5	R@10
Without Expansion	35.7	57.3	69.4	74.3
With Expansion (Baseline)	36.2	64.1	80.0	83.4

Candidate event count trade-off. To examine whether SAG’s gains simply result from expanding the candidate pool, we investigate the effect of varying K_{cand} (Table 5). When the number of candidate events increases from 50 to 100, Recall@5 improves from 76.1% to 80.0%, a significant gain. Further expanding to 200 or 500 yields sharply diminishing marginal return: the marginal gain falls below 0.089 Recall@5 points per million additional input tokens. We therefore set $K_{\text{cand}} = 100$ as the default operating point, balancing recall quality against LLM invocation cost.

Table 5: Trade-off between candidate event count and token cost.

Number of Events	MuSiQue				Tokens (M)
	R@1	R@2	R@5	R@10	
50	36.1	61.6	76.1	79.7	12.0
100 (Baseline)	36.2	64.1	80.0	83.4	20.0
200	36.5	65.0	80.9	84.4	35.5
500	36.3	64.3	81.8	86.1	76.4

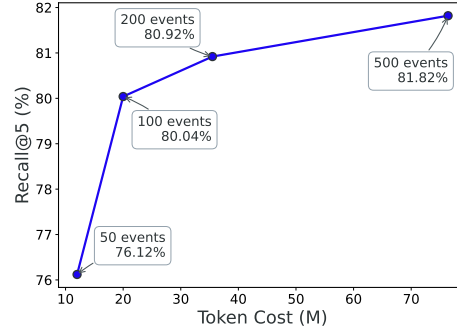


Figure 3: Token marginal benefit analysis

Necessity of the LLM for reranking. Replacing the LLM with the lightweight Qwen3-Reranker-0.6B causes Recall@5 on MuSiQue to drop from 80.0% to 62.2%, a decline of 17.8 percentage points (Table 6). A lightweight reranker scores each candidate independently and cannot jointly assess which subsets of events collectively form a complete reasoning chain. Given the compressed candidate set, the LLM can detect which events share entities and how they depend on one another logically, and thus identify more accurately the subset of events critical to the current query.

Table 6: Comparison between lightweight reranker and LLM filtering in the final selection stage (MuSiQue).

Method	MuSiQue			
	R@1	R@2	R@5	R@10
Qwen3-Reranker-0.6B	32.5	46.7	62.2	70.8
Qwen3.6-Flash	36.2	64.1	80.0	83.4

Complementarity of dual-path candidate pools. The candidate pool combines a structural path (events reranked by LLM and mapped to chunks) and a semantic path (chunks retrieved directly by the query vector). We vary the allocation between the two paths while fixing the total candidate count at $K_{\text{out}} = 10$, as shown in Table 7. When $K_{\text{event}} = 0$, the system degenerates to pure semantic retrieval, with Recall@5 at only 56.2%; with $K_{\text{event}} = 2$, it rises to 73.3%; at $K_{\text{event}} = 5$ (baseline), it reaches the optimum of 80.0%. The fact that even a few event-derived candidates produce a substantial gain indicates that the structural path supplies cross-entity reasoning evidence the semantic path misses, and that the two paths are highly complementary rather than redundant in their evidence coverage.

Table 7: LLM-based event selection vs. direct chunk-based selection (MuSiQue).

LLM Event	Query chunk	MuSiQue			
		R@1	R@2	R@5	R@10
0	10	29.4	41.6	56.2	63.8
2	8	36.6	64.5	73.3	77.6
4	6	36.6	63.6	79.6	82.6
5 (Baseline)	5	36.2	64.1	80.0	83.4

5.3 ROBUSTNESS TO THE EMBEDDING MODEL

To verify that SAG’s retrieval gains stem from structural design rather than a specific embedding model, we replace BGE-Large-EN-v1.5 with the stronger NV-Embed-v2 as the underlying embedding model, re-running both SAG and HippoRAG 2 on MuSiQue. As shown in Table 8, with NV-Embed-v2, SAG achieves Recall@5 of 81.7%, vs. HippoRAG 2’s 74.6%; when we switch back to BGE-Large-EN-v1.5, SAG remains stable at 80.0%, while HippoRAG 2 drops to 65.1%. HippoRAG 2 loses nearly 10 percentage points; SAG is barely affected. This asymmetric impact directly reflects the mechanistic difference between the two methods. HippoRAG 2’s multi-hop retrieval centers on Personalized PageRank propagation, with initial scores assigned by embedding similarity; changes in embedding quality are amplified hop by hop along the propagation path, ultimately producing a large effect on retrieval results. SAG’s structural gains come primarily from SQL joins, which are based on exact string matching and are independent of embedding quality. Even with the weaker BGE-Large-EN-v1.5, SAG leads HippoRAG 2 at 80.0% vs. 65.1%, demonstrating that SAG’s core advantage does not depend on a stronger embedding model but stems from the architectural design itself.

Table 8: Retrieval results of SAG and HippoRAG 2 on MuSiQue under the NV-Embed-v2 configuration.

Method	MuSiQue			
	R@1	R@2	R@5	R@10
SAG (NV-Embed-v2)	36.4	64.6	81.7	86.6
HippoRAG 2 (NV-Embed-v2)	33.7	56.0	74.6	83.2

6 DISCUSSION

SAG’s retrieval gains come mainly from event-level semantic preservation and the structured connectivity that shared entities provide, not from making any single component stronger. The current method has several limitations, discussed below.

Pruning mechanisms. The current entity frontier pruning budget and similarity thresholds are set empirically on the development set. Low-frequency yet critical bridging entities may be truncated early during expansion; this risk is particularly pronounced in long entity chain scenarios, and is consistent with SAG’s Recall@5 on 2WikiMultiHop being slightly below HippoRAG 2’s. Future work can stratify entities by corpus frequency, allocating larger expansion budgets to low-frequency entities, or can introduce failure-case analysis to automatically calibrate thresholds.

Entity merging. SAG employs a lightweight entity strategy, performing only string normalization and SQL deduplication before ingestion, so that chunk processing can proceed independently and concurrently, supporting incremental writes. The trade-off is that different string forms of the same entity may be treated as distinct index points, weakening cross-document connection density. Introducing a lightweight entity alias table is a feasible direction; it could consolidate synonymous forms without sacrificing write independence.

Event-level updates for agent memory. Currently, SAG can organize memory as an appendable, retrievable event index, supporting continuous ingestion of new events. In agent memory scenarios, preference overriding, task status changes, and similar situations further require the system to update,

invalidate, or preserve historical versions of existing events, not merely append. Extending the SAG index into a versioned agent memory substrate is the natural next step, and is the main direction we leave for future work.

7 CONCLUSION

This paper presents SAG, a structured retrieval framework for retrieval and agent systems. SAG employs SQL as its structural engine, a vector database as its semantic engine, with event-entity indexing and query-time dynamic hyperedges as its core mechanisms. Its design principle is not to replace standard RAG with stronger modules, but to redistribute responsibilities across the retrieval pipeline. SQL delimits candidate boundaries and enforces structural organization, the event-entity index carries complete semantic units, and dynamic hyperedges recover local higher-order relations at query time. Vector models and LLMs intervene only at the highest-value points, rather than dominating the entire pipeline. Under a unified underlying configuration, SAG achieves the best result on 8 out of 9 Recall@K metrics across three multi-hop benchmarks. Ablation analysis further demonstrates that this advantage does not stem from stronger components but is rooted in the structural organization itself. What SAG changes, therefore, is not an isolated component of standard RAG, but the organizational paradigm as data enters retrieval and agent systems. Building on this, extending the event-entity index into a versioned, temporally-aware agent memory substrate is the natural direction for future work.

REFERENCES

- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=hSyW5go0v8>.
- Wenhu Chen, Hanwen Zha, Zhiyu Chen, Wenhan Xiong, Hong Wang, and William Yang Wang. HybridQA: A dataset of multi-hop question answering over tabular and textual data. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020. URL <https://arxiv.org/abs/2004.07347>.
- Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024. URL <https://arxiv.org/abs/2404.16130>.
- Bahare Fatemi, Perouz Taslakian, David Vazquez, and David Poole. Knowledge hypergraphs: Prediction beyond binary relations. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI)*, 2020. URL <https://arxiv.org/abs/1906.00137>.
- Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. Hypergraph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019. URL <https://arxiv.org/abs/1809.09401>.
- Mikhail Galkin, Priyansh Trivedi, Gaurav Maheshwari, Ricardo Usbeck, and Jens Lehmann. Message passing for hyper-relational knowledge graphs. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. URL <https://arxiv.org/abs/2009.10847>.
- Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. LightRAG: Simple and fast retrieval-augmented generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. URL <https://arxiv.org/abs/2410.05779>.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2405.14831>.

-
- Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to memory: Non-parametric continual learning for large language models. In *International Conference on Machine Learning (ICML)*, 2025. URL <https://arxiv.org/abs/2502.14802>.
- Xiaoxin He, Yijun Tian, Yifei Sun, Nitesh V. Chawla, Thomas Laurent, Yann LeCun, Xavier Bresson, and Bryan Hooi. G-retriever: Retrieval-augmented generation for textual graph understanding and question answering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2402.07630>.
- Jonathan Herzig, Pawel Krzysztof Nowak, Thomas Müller, Francesco Piccinno, and Julian Eisen-schlos. TaPas: Weakly supervised table parsing via pre-training. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020. URL <https://aclanthology.org/2020.acl-main.398/>.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics (COLING)*, 2020. URL <https://aclanthology.org/2020.coling-main.580/>.
- Chenxu Hu, Jie Fu, Chenzhuang Du, Simian Luo, Junbo Zhao, and Hang Zhao. ChatDB: Augmenting LLMs with databases as their symbolic memory. *arXiv preprint arXiv:2306.03901*, 2023. URL <https://arxiv.org/abs/2306.03901>.
- Haoyu Huang, Yongfeng Huang, Junjie Yang, Zhenyu Pan, Yongqiang Chen, Kaili Ma, Hongzhi Chen, and James Cheng. Retrieval-augmented generation with hierarchical knowledge. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025. URL <https://arxiv.org/abs/2503.10150>.
- Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024. URL <https://aclanthology.org/2024.naacl-long.389/>.
- Jinhao Jiang, Kun Zhou, Zican Dong, Keming Ye, Wayne Xin Zhao, and Ji-Rong Wen. StructGPT: A general framework for large language model to reason on structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023a. URL <https://arxiv.org/abs/2305.09645>.
- Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023b. URL <https://aclanthology.org/2023.emnlp-main.495/>.
- Vladimir Karpukhin, Barlas Öguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. URL <https://aclanthology.org/2020.emnlp-main.550/>.
- Chankyu Lee, Rajarshi Roy, Mengyao Xu, Jonathan Raiman, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. NV-Embed: Improved techniques for training LLMs as generalist embedding models. *arXiv preprint arXiv:2405.17428*, 2024. URL <https://arxiv.org/abs/2405.17428>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL <https://arxiv.org/abs/2005.11401>.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C.C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM already serve as a database interface? A BIG bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL <https://arxiv.org/abs/2305.03111>.

-
- Zhuoshi Li, Xin-Cheng Chen, Huiqian Yu, Haiming Lin, Jingbo Shang, Qiang Tang, Furu Wei, Xuancheng Ren, Longtao Huang, and Chao Li. StructRAG: Boosting knowledge intensive reasoning of LLMs via inference-time hybrid information structurization. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2410.08815>.
- Hao Liu, Zhengren Wang, Xi Chen, Zhiyu Li, Feiyu Xiong, Qinhan Yu, and Wentao Zhang. HopRAG: Multi-hop reasoning for logic-aware retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025. URL <https://arxiv.org/abs/2502.12442>.
- Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. TAPEX: Table pre-training via learning a neural SQL executor. In *International Conference on Learning Representations (ICLR)*, 2022. URL <https://arxiv.org/abs/2107.07653>.
- Haoran Luo, Haihong E, Guanting Chen, Qika Lin, Yikai Guo, Fangzhi Xu, Zemin Kuang, et al. Graph-R1: Towards agentic GraphRAG framework via end-to-end reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2026. URL <https://arxiv.org/abs/2507.21892>.
- Vaibhav Mavi, Anubhav Jangra, and Adam Jatowt. Multi-hop question answering. *Foundations and Trends in Information Retrieval*, 17(5):457–586, 2024. URL <https://arxiv.org/abs/2204.09140>.
- Costas Mavromatis, Soji Adeshina, Vassilis N. Ioannidis, Zhen Han, Qi Zhu, Ian Robinson, Bryan Thompson, Huzefa Rangwala, and George Karypis. BYOKG-RAG: Multi-strategy graph retrieval for knowledge graph question answering. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025. URL <https://arxiv.org/abs/2507.04127>.
- Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/72223cc66f63calaa59edaec1b3670e6-Abstract-Conference.html.
- Qwen Team. Qwen3 technical report. Technical report, Alibaba Group, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2401.18059>.
- Jiashuo Sun, Chengjin Xu, Luminyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2307.07697>.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics (TACL)*, 10, 2022. URL <https://aclanthology.org/2022.tacl-1.31/>.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. URL <https://aclanthology.org/2023.acl-long.557/>.
- Changjian Wang, Weihong Deng, Weili Guan, Quan Lu, and Ning Jiang. Cross-granularity hypergraph retrieval-augmented generation for multi-hop question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026. URL <https://arxiv.org/abs/2508.11247>.
- Yu Wang, Nedim Lipka, Ryan A. Rossi, Alexa Siu, Ruiyi Zhang, and Tyler Derr. Knowledge graph prompting for multi-document question answering. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024. URL <https://arxiv.org/abs/2308.11730>.

-
- Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-pack: Packaged resources to advance general chinese embedding. *arXiv preprint arXiv:2309.07597*, 2023. URL <https://arxiv.org/abs/2309.07597>.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018. URL <https://aclanthology.org/D18-1259/>.
- Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018. URL <https://aclanthology.org/D18-1425/>.
- Nan Zhang, Prafulla Kumar Choubey, Alexander Fabbri, Gabriel Bernadett-Shapiro, Rui Zhang, Prasenjit Mitra, Caiming Xiong, and Chien-Sheng Wu. SiReRAG: Indexing similar and related information for multihop reasoning. In *International Conference on Learning Representations (ICLR)*, 2025. URL <https://openreview.net/forum?id=yp95goUAT1>.
- Dengyong Zhou, Jiayuan Huang, and Bernhard Schölkopf. Learning with hypergraphs: Clustering, classification, and embedding. In *Advances in Neural Information Processing Systems (NIPS)*, 2006.
- Ziyuan Zhuang, Zhiyang Zhang, Sitao Cheng, Fangkai Yang, Jia Liu, Shujian Huang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. EfficientRAG: Efficient retriever for multi-hop question answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3392–3411, 2024. URL <https://arxiv.org/abs/2408.04259>.